

1 · Why Function Approximation? The Scalability Problem

The Problem with Tabular Methods

Tabular RL stores one value per state (or state-action pair). This breaks down when the state space is large or continuous:

- **Atari games:** 210×160 pixels, 128 colours each, 4 stacked frames $\Rightarrow 128^{210 \times 160 \times 4}$ possible states — impossible to store or visit them all.
- **Robotics:** joint angles and velocities are continuous — infinitely many states.
- **Tabular Q-table:** each state entry is independent — learning $Q(s, a)$ teaches us *nothing* about nearby states s' (no generalisation).

Solution — Function Approximation: represent the value function as a parameterised function

$$\hat{v}(s; \mathbf{w}) \approx v_{\pi}(s) \quad \text{or} \quad \hat{q}(s, a; \mathbf{w}) \approx q_{\pi}(s, a)$$

where $\mathbf{w} \in \mathbb{R}^d$ is a weight vector with $d \ll |\mathcal{S}|$. Changing one weight changes the estimated value of *many* states — **generalisation**.

Feature Engineering: hand-craft a feature vector $\phi(s) \in \mathbb{R}^d$ that captures the important structure of the state.

Type	Form	Properties
Linear FA	$\hat{v}(s; \mathbf{w}) = \mathbf{w}^{\top} \phi(s)$	Simple; guaranteed convergence; limited expressive power
Neural Network	$\hat{v}(s; \theta) = f_{\theta}(s)$	Very expressive; can learn features; unstable without tricks

2 · Notion of Semi-Gradient

Recall supervised learning: given a fixed target y , minimise the squared error:

$$L(\mathbf{w}) = [y - \hat{v}(s; \mathbf{w})]^2 \quad \Rightarrow \quad \mathbf{w} \leftarrow \mathbf{w} + \alpha [y - \hat{v}(s; \mathbf{w})] \nabla_{\mathbf{w}} \hat{v}(s; \mathbf{w})$$

In RL: the target itself depends on $\mathbf{w}$ (it is bootstrapped):

$$y_{\text{TD}} = R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w})$$

Semi-Gradient TD Update

We *pretend* the target is fixed — ignore its dependence on $\mathbf{w}$ when computing gradients:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \underbrace{[R_{t+1} + \gamma \hat{v}(S_{t+1}; \mathbf{w}) - \hat{v}(S_t; \mathbf{w})]}_{\delta_t} \cdot \nabla_{\mathbf{w}} \hat{v}(S_t; \mathbf{w})$$

The gradient $\nabla_{\mathbf{w}} \hat{v}(S_{t+1}; \mathbf{w})$ is *not* included — hence **semi-gradient**.

Why “semi”? A true gradient update would differentiate *both* $\hat{v}(S_t; \mathbf{w})$ and $\hat{v}(S_{t+1}; \mathbf{w})$ with respect to $\mathbf{w}$. Semi-gradient only differentiates through the *prediction* (S_t), not the *target* (S_{t+1}). This makes the algorithm computationally tractable but means it is no longer following a true gradient — convergence is not guaranteed for non-linear FA (though it works well in practice with tricks like experience replay and target networks).

For linear FA: $\nabla_{\mathbf{w}} \hat{v}(s; \mathbf{w}) = \phi(s)$, so:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t \phi(S_t)$$

3 · DQN — Deep Q-Network (Mnih et al., Nature 2015)

Goal: Apply Q-Learning with a deep neural network as the function approximator. Naively replacing the Q-table with an NN is *unstable* — three problems arise:

Three Problems with Naive Deep Q-Learning

1. **Correlated samples:** consecutive frames in a game are highly similar. Gradient updates based on them are nearly identical — the network overfits one sequence.
2. **Non-stationary targets:** the target $R + \gamma \max_{a'} Q(s', a'; \theta)$ changes every step as θ updates — like chasing a moving goal post.
3. **Divergence:** without controls, the network weights can grow unboundedly.

DQN's Two Key Innovations

Innovation 1 — Experience Replay (fixes problem 1):

- Store transitions $(S_t, A_t, R_{t+1}, S_{t+1})$ in a **replay buffer** $\mathcal{D}$.
- At each update step, sample a random **mini-batch** from $\mathcal{D}$.
- Random sampling breaks temporal correlations between samples.
- Each transition can be used multiple times (data efficiency).

Innovation 2 — Target Network (fixes problem 2):

- Maintain two networks: **online** $Q(s, a; \theta)$ (updated every step) and **target** $Q(s, a; \theta^-)$ (updated every C steps by copying θ).
- The target for the loss is computed using the frozen θ^- :

$$y_i = R_i + \gamma \max_{a'} Q(s'_i, a'; \theta^-)$$

- Target is fixed for C steps — the goal post stays still.

DQN Loss function (minimised over mini-batches sampled from $\mathcal{D}$):

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]$$

DQN Algorithm (Nature 2015 — key steps):

1. Initialise replay buffer $\mathcal{D}$, online network θ (random), target network $\theta^- \leftarrow \theta$.
2. For each step: observe s ; select a via ϵ -greedy on $Q(s, \cdot; \theta)$.
3. Execute a , observe r, s' ; store (s, a, r, s') in $\mathcal{D}$.
4. Sample random mini-batch from $\mathcal{D}$; compute targets y_i .
5. Gradient descent on $L(\theta)$ to update θ .
6. Every C steps: $\theta^- \leftarrow \theta$ (update target network).

Input representation (Atari): 4 consecutive game frames stacked, grayscaled, resized to 84×84 — fed into a 3-layer CNN, then fully connected layers outputting $|\mathcal{A}|$ Q-values simultaneously.

4 · DDQN — Double Deep Q-Network (van Hasselt et al., 2015)

Motivation: DQN still suffers from **maximization bias** (see Session 9). Even with a target network, the target uses $\max_{a'} Q(s', a'; \theta^-)$ — the same network selects *and* evaluates the best action, leading to systematic overestimation.

DDQN Fix — Decouple Action Selection from Evaluation

DQN target:

$$y^{\text{DQN}} = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

DDQN target: Use online network θ to *select* the action; target network θ^- to *evaluate* it:

$$y^{\text{DDQN}} = r + \gamma Q\left(s', \underbrace{\arg \max_{a'} Q(s', a'; \theta)}_{\text{online network selects}}; \underbrace{\theta^-}_{\text{target network evaluates}}\right)$$

Why this helps: The online and target networks are different (updated at different rates). If the online network overestimates action a^* , the target network (with its own independent weights) is unlikely to also overestimate the same action. The overestimation is no longer systematic.

Only change from DQN: one line in the target computation — everything else is identical. This makes DDQN easy to implement on top of an existing DQN.

	DQN	DDQN
Action selection	θ^-	θ (online)
Action evaluation	θ^-	θ^- (target)
Maximization bias	Yes	Reduced
Performance	Strong baseline	Consistently better; less overestimation

5 · RAINBOW (High Level — Not for Evaluation)

RAINBOW — Combining 6 Improvements

RAINBOW (Hessel et al., 2017) shows that combining multiple independent DQN improvements gives far better results than any single improvement.

The 6 components:

1. **Double DQN** — reduces maximization bias
2. **Prioritised Experience Replay** — sample transitions with higher TD error more often
3. **Dueling Networks** — separate streams for $V(s)$ and $A(s, a)$; better state-value estimation
4. **Multi-step Returns** — use n -step TD targets instead of 1-step (less bias)
5. **Distributional RL** — learn the full distribution of returns, not just the mean
6. **Noisy Networks** — learned noise in weights replaces ε -greedy exploration

Awareness only — details not required for evaluation.

6 · Numerical Example — Linear FA: SARSA vs Q-Learning vs Double Q-Learning

Setup

State: s with feature $\phi(s) = 1$ (scalar, linear approximator).

Actions: Left (L) or Right (R). Weights: w_L, w_R so that $Q(s, L; \mathbf{w}) = w_L \cdot 1 = w_L$ and $Q(s, R; \mathbf{w}) = w_R \cdot 1 = w_R$.

Initial values: $w_L = 2.0, w_R = 1.0$ (greedy action at s is **L**).

Transition: Agent takes action **R** (exploratory, ε -greedy) from s , receives reward $R = 1$, arrives at s' (same features: $\phi(s') = 1$, same $w_L = 2.0, w_R = 1.0$).

Next action (SARSA, ε -greedy): chooses **R** (another exploratory step).

Hyperparameters: $\alpha = 0.1, \gamma = 0.9$.

Method 1 — SARSA: on-policy / actual next action

$$\text{TD target} = R + \gamma \cdot Q(s', A'; \mathbf{w}) = 1 + 0.9 \times w_R = 1 + 0.9 \times 1.0 = 1.9$$

$$\delta = \text{target} - Q(s, R; \mathbf{w}) = 1.9 - w_R = 1.9 - 1.0 = 0.9$$

$$w_R \leftarrow w_R + \alpha \cdot \delta \cdot \phi(s) = 1.0 + 0.1 \times 0.9 \times 1 = \mathbf{1.09}$$

SARSA used $Q(s', R) = 1.0$ (the *actual* next action, even though L is greedy). Result is conservative.

Method 2 — Q-Learning: off-policy / greedy max action

$$\max_{a'} Q(s', a') = \max(w_L, w_R) = \max(2.0, 1.0) = 2.0 \quad (\text{action L wins})$$

$$\text{TD target} = R + \gamma \times 2.0 = 1 + 0.9 \times 2.0 = 2.8$$

$$\delta = 2.8 - Q(s, R; \mathbf{w}) = 2.8 - 1.0 = 1.8$$

$$w_R \leftarrow 1.0 + 0.1 \times 1.8 \times 1 = \mathbf{1.18}$$

Q-Learning used $Q(s', L) = 2.0$ (the *greedy* action, not actually taken). More optimistic than SARSA — classic maximization bias in action.

Method 3 — Double Q-Learning (two estimators)

Maintain $Q^{(1)}$ and $Q^{(2)}$. Initial weights: $w_L^{(1)} = 2.0$, $w_R^{(1)} = 1.0$ and $w_L^{(2)} = 1.8$, $w_R^{(2)} = 0.8$.

Update $Q^{(1)}$ this step:

$$a^* = \arg \max_{a'} Q^{(1)}(s', a') = \arg \max(w_L^{(1)}, w_R^{(1)}) = L \quad (2.0 > 1.0)$$

$$\text{TD target} = R + \gamma \cdot Q^{(2)}(s', a^*) = 1 + 0.9 \times w_L^{(2)} = 1 + 0.9 \times 1.8 = 2.62$$

$$\delta = 2.62 - Q^{(1)}(s, R) = 2.62 - 1.0 = 1.62$$

$$w_R^{(1)} \leftarrow 1.0 + 0.1 \times 1.62 \times 1 = \mathbf{1.162}$$

$Q^{(1)}$ selected action L; $Q^{(2)}$ evaluated it at 1.8 (lower than $Q^{(1)}$'s estimate of 2.0). Less overestimation than plain Q-Learning.

Comparison — Three Methods

Method	Target	New w_R	Reason for difference
SARSA	1.9	1.09	Used actual next action R ($Q = 1.0$) — conservative
Q-Learning	2.8	1.18	Used max action L ($Q = 2.0$) — most optimistic
Double Q-Learning	2.62	1.162	$Q^{(1)}$ selected L; $Q^{(2)}$ evaluated at 1.8 — less biased

Key takeaway: SARSA < Double Q-Learning < Q-Learning (in terms of optimism and new w_R).

7 · Exam Key-Takeaways — Sessions 11 & 12

Must Know — Value-Based Deep RL

1. **Why FA:** tabular methods can't scale to large/continuous state spaces (Atari: astronomically large).
2. **Linear FA:** $\hat{v}(s; \mathbf{w}) = \mathbf{w}^\top \phi(s)$; generalises across states via shared weights.
3. **Semi-gradient:** treat the bootstrapped target as fixed; only differentiate through the prediction. Update: $\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t \nabla \hat{v}(S_t; \mathbf{w})$.
4. **DQN's two innovations:** (a) **Experience Replay** — random mini-batch from buffer breaks correlations; (b) **Target Network** (θ^-) — frozen for C steps, stabilises targets.
5. **DQN target:** $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$.
6. **DDQN target:** $y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta) ; \theta^-)$. Online θ selects, target θ^- evaluates.
7. **DDQN vs DQN:** one line changes in the target — but significantly reduces overestimation.
8. **Numerical example:** SARSA is conservative (actual next action), Q-Learning is optimistic (max action), Double Q-Learning is between (cross-estimate evaluation).
9. **RAINBOW:** combines 6 improvements; not for evaluation — awareness only.

DRL EC3 Exam Handout — Sessions 11 & 12 (Value-Based Deep RL). Ref: S&B Ch. 9,16; Mnih et al. 2013, 2015; van Hasselt et al. 2015.